

OBJECT-ORIENTED PROGRAMMING

BBF - 203E

Exercise - 4

Course Instructors:
Cihan TOPAL
Feza BUZLUCA
Sanem KABADAYI

Prepared by: Kuzey ARAR
arar25@itu.edu.tr

Attention is All You Need

Abstract: The dominant sequence transduction models are based on complex recurrent or convolutional neural networks that include an encoder and a decoder. The best performing models also connect the encoder and decoder through an attention mechanism. We propose a new simple network architecture, the Transformer, based solely on attention mechanisms, dispensing with recurrence and convolutions entirely. Experiments on two machine translation tasks show these models to be superior in quality while being more parallelizable and requiring significantly less time to train. Our model achieves 28.4 BLEU on the WMT 2014 English-to-German translation task, improving over the existing best results, including ensembles, by over 2 BLEU. On the WMT 2014 English-to-French translation task, our model establishes a new single-model state-of-the-art BLEU score of 41.8 after training for 3.5 days on eight GPUs, a small fraction of the training costs of the best models from the literature. We show that the Transformer generalizes well to other tasks by applying it successfully to English constituency parsing both with large and limited training data.

Above is the abstract for **Attention Is All You Need (2017)** paper which is the foundation architecture of Large Language Models. While this exercise does not represent real transformer or modern language models in any sort, I hope it will spark your curiosity how these models work on the lowest level.

EXERCISE 4: Composition/ Aggregation Structure in Attention

You are tasked with completing file structures of .cpp files of related .h files. Attention mechanism and needed functions for computing attentions are provided in the source codes. This exercise's goal is to show you composition and aggregation relation between class objects. Constructor implementations in .cpp files are left blank as TODO sections for your contributions.

1. Header Files Structures:

1.1 InputSequence.h : Acts as the container for the sequence of token embeddings. Token embeddings are hard-coded.

1.2 ProjectionWeights.h : It is responsible for transforming the standard input sequence into the specific Queries (Q), Keys (K), and Values (V) needed for a single attention head.

1.3 AttentionHead.h : Performs the actual scaled dot-product attention calculation using its own specific set of weights.

1.4 MultiHeadAttention.h : It groups multiple independent AttentionHead objects together. In a real Transformer, it would execute them in parallel and concatenate their results. Our implementation will not be in parallel.

2. Expected Output

```
Head 0 Output Matrix (Context):  
[ 0.761433 0.798134 ]  
[ 0.768678 0.805729 ]  
Head 1 Output Matrix (Context):  
[ 1.43202 1.46638 ]  
[ 1.48779 1.52349 ]  
Destructor called for Attention Head.  
Destructor: Freeing Q, K, V arrays.  
Destructor called for Attention Head.  
Destructor: Freeing Q, K, V arrays.
```

3. Requirements

- Don't modify any of the header files and main.cpp. You have all the needed functions defined for you.
- Be sure to deallocate when dynamically allocating memory.

4. References

- Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., ... & Polosukhin, I. (2017). Attention is all you need. *Advances in neural information processing systems*, 30.